

Reviewer Pack — Mahler Measure of Signed-Degree Polynomial Bounds $\text{ACC}^0[6]$ Size

Entry a8a04f589e9b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-26 23:26:40 UTC

Mahler Measure of Signed-Degree Polynomial Bounds $\text{ACC}^0[6]$ Size

Entry ID: a8a04f589e9b **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-26 23:26:40 UTC

1. Conjecture

Field A (mathematical branch): Algebraic number theory (Mahler measure / Weil height of integer polynomials) **Field B** (complexity object): $\text{ACC}^0[6]$ circuit size for CNF-represented Boolean functions

Statement:

For any CNF F on n variables with $m \leq \text{poly}(n)$ clauses, define the signed-degree polynomial $P_F(z) = z^{n+1} + \sum_{i=1}^m d_i z^{i-1}$, where $d_i = (\#\text{clauses containing literal } x_i) - (\#\text{clauses containing } \neg x_i)$. Let $M(P_F) = \prod_{|\alpha| > 1} |\alpha|$ be the Mahler measure of P_F (over its complex roots α), and let $s_{\text{ACC}}(F)$ be the minimum size of an $\text{ACC}^0[6]$ circuit computing the function f_F encoded by F . Then there exists an absolute constant $c > 0$ such that for all such F , $\log s_{\text{ACC}}(F) \geq c \cdot \log M(P_F) / \log n$; equivalently, every $\text{ACC}^0[6]$ circuit of size s computing f_F forces $\log M(P_F) \leq (\log n)(\log s)/c$.

Rationale (proposer's reasoning):

Williams' route to $\text{NEXP} \not\subseteq \text{ACC}^0$ exploits the fact that ACC^0 functions admit short multilinear-polynomial-modulo- m representations whose roots, viewed at primitive roots of unity, cluster on the unit circle; the Mahler measure of an integer polynomial is precisely the multiplicative height that detects deviation of roots from the unit circle. Signed-degree statistics of a CNF survive any size-preserving $\text{ACC}^0[6]$ equivalent encoding, so a CNF whose signed-degree polynomial has roots far from $|z|=1$ should not admit small mod-6 cancellations. The Mahler-measure $\times \text{ACC}^0$ bridge appears in <5 papers (it shows up in dynamical entropy and Lehmer's problem, almost never in complexity).

Taxonomy category: $\text{ACC_LB_via_WILLIAMS}$ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 20eee795b1d94f0d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across $n \in \{8, 10, 12, 14, 16, 18, 20\}$ with 3 seeds per random/Tseitin family, compute $R(F) = \log M(P_F) / \log n$. SUPPORTED if mean $R_EASY \leq 1.0$ for all n AND mean R_HARD (Tseitin) grows monotonically with slope ≥ 0.3 per unit $\log n$. FALSIFIED if any Tseitin seed yields $R < 0.4$ stable across ≥ 3 consecutive n , or any easy family yields $R > 2$ sustained across ≥ 3 consecutive n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.85	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.80	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Mahler measure Boolean function circuit complexity lower bound - ACC0 circuit lower bound CNF polynomial height algebraic-Weil height integer polynomial circuit complexity CNF clauses

Top relevant hits considered: - [<http://arxiv.org/abs/1806.00355v1>] Mahler's work on Diophantine equations and subsequent developments - [<http://arxiv.org/abs/1806.00356v1>] Mahler's work on the geometry of numbers - [<http://arxiv.org/abs/1701.06097v1>] Graph complexity and Mahler measure - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/2502.03039v2>] Lower bounds for heights on some algebraic dynamical systems - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/math/0410111v2>] Integer Polynomial Optimization in Fixed Dimension

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            if not any(clause[i] == -clause[j] for i in range(n) for j in range(i + 1, n)):
                clauses.append(clause)
        return clauses
```

```

def generate_easy_cnf(n):
    return [(i + 1)] for i in range(n)]

def generate_medium_cnf(n):
    return [generate_cnf(n, 4) for _ in range(3)]

def generate_hard_cnf(n):
    # Placeholder for actual hard CNF generation
    return generate_cnf(n, n)

def sign_degree_polynomial(clauses, n):
    d = [0] * (n + 1)
    m = len(clauses)
    for clause in clauses:
        for literal in clause:
            if literal > 0:
                d[literal - 1] += 1
            else:
                d[-literal - 1] -= 1
    return [m] + d

def mahler_measure(coefficients):
    n = len(coefficients) - 1
    roots = np.roots(coefficients)
    measure = 1.0
    for root in roots:
        if abs(root) > 1:
            measure *= abs(root)
    return measure

def log_ratio(n, m):
    coefficients = sign_degree_polynomial(generate_cnf(n, m), n)
    M = mahler_measure(coefficients)
    return math.log(M) / math.log(n)

n_values = [8, 10, 12, 14]
easy_r = [log_ratio(n, 2 * n) for n in n_values]
medium_r = [log_ratio(n, 3 * n) for n in n_values]
hard_r = [log_ratio(n, n) for n in n_values]

mean_easy_r = sum(easy_r) / len(easy_r)
mean_medium_r = sum(medium_r) / len(medium_r)
mean_hard_r = sum(hard_r) / len(hard_r)

conjecture_holds = (mean_easy_r <= 1.0 and
                    all(mean_hard_r >= 0.3 * math.log(n + 1) / math.log(n) for n in n_values))

counterexample = "" if conjecture_holds else "easy_family"

return {
    "metric_name": "log_ratio",
    "metric_value": mean_easy_r,
    "instances_tested": len(easy_r),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_easy_r = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_easy_r} std=0.0 support_fraction=1.0")
    elif any(r["counterexample"] == "easy_family" and r["instances_tested"] >= 3 for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["counterexample"] == "easy_family")
        print(f"RESULT: FALSIFIED counterexample=\"easy_family\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1882f1ed.py", line 91, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1882f1ed.py", line 65, in run_trial
    easy_r = [log_ratio(n, 2 * n) for n in n_values]
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1882f1ed.py", line 61, in log_ratio
    M = mahler_measure(coefficients)
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1882f1ed.py", line 52, in mahler_measure
    roots = np.roots(coefficients)
             ^^
NameError: name 'np' is not defined. Did you mean: 'n'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `NameError` (`'np'` not defined) before producing any data, so neither the support nor falsification conditions could be evaluated. | next: Fix the missing `'import numpy as np'` in the test script and rerun the $n \in \{8, \dots, 20\}$ sweep across both random and Tseitin families.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	95383
2	preregistration	claude_max	opus	0	0	6848
3	novelty	claude_max	opus	0	0	3116
4	novelty	claude_max	opus	0	0	10350
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30287
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10378
7	judge	claude_max	opus	0	0	4032

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160396 ms total latency. Provider mix: `{'claude_max': 5, 'ollama_remote': 2}`

(full prompt+response transcripts available in `research/audit/a8a04f589e9b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a8a04f589e9b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a8a04f589e9b.tar.gz` (if generated)